Recommendo: A Movie Recommendation WebApp

ML Engineer
Davide Zorzetto
DAVIDE.ZORZETTO@studenti.units.it

June 27, 2025

ABSTRACT

In this report, we present **Recommendo**, a machine learning-based web application for personalized movie recommendations. The site allows users to select five films they already enjoy and, through data-driven techniques, a recommendation engine computes movie similarities and generates personalized suggestions.

Introduction & Problem Statement

In a era where users must choose between dozens of digital streaming platforms, they are often overwhelmed by the sheer volume of available films and series. This has led to a sort of paradox of choice, where the abundance of possibilities brought onto users decision fatigue.
Another key point is that, while there are tradition recommendation systems embedded directly into these services, they are not always transparent or personalized, particularly when users' history is not present.
Our proposal, Recommendo, has been built with these challenges in mind: we allow users to select up to 10 (at least one) films they already like and then return tailored recommendations based on those choices. The core of our project is a data-driven, machine learning-powered recommendation pipeline. As a machine learning engineer, my role involved designing and implementing models, testing their performance, and collaborating closely with the data engineer to maximize the effectiveness of the system.

Background

The development of our project required several things: we need to find coordination between modern web development practices, software design methodologies and data-driven machine learning components.
At the beginning of the project, we decided to adopt an Agile software development methodology. We broke down the work into weekly milestones with iterative improvements. Task tracking, collaboration, and version control were managed using GitHub Projects, which helped structure issues, pull requests, milestones, and more.
Each team member worked within their dedicated branch (e.g., `dev-architecture` for the SE, `dev-ml` for the MLE, `dev-app` for the SD, and `dev-data` for the DE). Merges into the main branch occurred only via reviewed pull requests, which were verified by the software engineer.

Initially, we considered Streamlit for the frontend, due to its easy integration with Python-based ML models. However, early in the development, the software developer brought up during a meeting that he felt it was too limited for the task at hand, and that he could do better if we transitioned to a more flexible stack: the Flask backend was built using Flask, to make the integration with python smoother while the frontend was built using HTML, CSS and JavaScript.
As the ML engineer, I had to select models that could operate effectively with minimal user input (1–10 film titles), deliver low-latency responses, and integrate seamlessly into a web-based pipeline. The models we considered and/or developed included:

- k-NN on one-hot encoded features
- k-NN on embedding-based features (SBERT)
- Variational Autoencoder (VAE) on sentence embeddings — later discarded due to poor performance

We used a public Netflix dataset from Kaggle, which contains metadata such as title, cast, genre, description, and rating. Preprocessing was a critical step to prepare the data for both classical ML and embedding-based models. The dataset management and preprocessing were handled by the data engineer, while their integration into the Flask backend required coordination with the software developer. The dataset we decided to use was a dataset containing Netflix's Movies and TV-series, found on Kaggle.

Methodology/Approach

Machine Learning Pipeline

The recommendation system was developed using a structured pipeline:

1. **Data Preprocessing**:
   In collaboration with the data engineer, we handled heterogeneous features as follows:
   - Categorical variables (`director`, `country`, `cast`, `listed_in`) were one-hot encoded
   - High-cardinality one-hot features (e.g., `cast`) were filtered to include only actors appearing in ≥ 5 productions

- Text features (`description`, `listed_in`, `cast`, `title`) were embedded using Sentence-BERT
- `duration` was converted to numerical format
- Numerical variables such as `release_year` and `rating` were normalized
- Non-informative variables (`index`, `date_added`) were dropped
- Observations with missing values (except in `director`, `cast`, and `country`) were removed

2. **Feature Engineering**:
   We created two distinct feature spaces:
   - *Sparse Representation*: Traditional one-hot encoding of metadata
   - *Dense Representation*: SBERT embeddings (`all-MiniLM-L6-v2`) of textual descriptions

   We assessed the quality of the embedded feature vectors using dimensionality reduction techniques such as `t-SNE` and `UMAP`.

3. **Model Development**:
   We implemented and evaluated three approaches:
   - *KNN (One-Hot)*: Traditional k-Nearest Neighbors using mean squared error on sparse features
   - *KNN (Embedded)*, also named `MLOVIE`: A variant of KNN that uses cosine similarity in the SBERT embedding space for textual features and scores numerical features with $\frac{1}{\text{RMSE}+1}$
   - *VAE*: Variational Autoencoder for latent space learning, later abandoned due to poor cluster separation in the latent space

   In the KNN models, we introduced a hyperparameter to weight the relevance of different features. This improved performance by reducing the influence of less informative features (e.g., `title`) and emphasizing more meaningful ones, such as movie genre.

4. **Model Integration**:
   - Designed a unified loading and prediction interface:

```
def load(self):
    """Load necessary data"""

def predict(IDX: list, k=5) -> np.ndarray:
    """Return top-k similar
    movie indices"""
```

   - To suggest similar movies across all the user-selected films, the `predict()` function first computes the closest movie for each input based on similarity score or distance. Then, it selects the movie with the highest overall score among those closest matches
   - A similar `fit()` interface was designed to compute the embeddings and store the resulting data
   - Collaborated with the Data Engineer to fine-tune the model hyperparameters.

MLOps Implementation

The machine learning lifecycle followed MLOps principles:

- **Version Control**: Models, datasets, and preprocessing pipelines versioned in GitHub
- **Reproducibility**: Frozen dependencies via requirements.txt and model serialization
- **Testing**: Unit tests for embedding consistency and prediction stability
- **Monitoring**: Latency tracking during development (sub-second response achieved)

Results

Performance Evaluation

Models were evaluated on four critical dimensions:

Metric	🟦	🟩
Inference Speed (5 movies input)	332 ms	232 ms
Memory Footprint	82 MB	62 MB
Cluster Coherence†	low	high
Human Evaluation†	medium	high

Table 1: 🟦 KNN (One-Hot) 🟩 KNN (Embedded)
†Human evaluation: 4 users rated relevance (4 test users)

The following plot shows how inference time increases as the number of input movies grows:

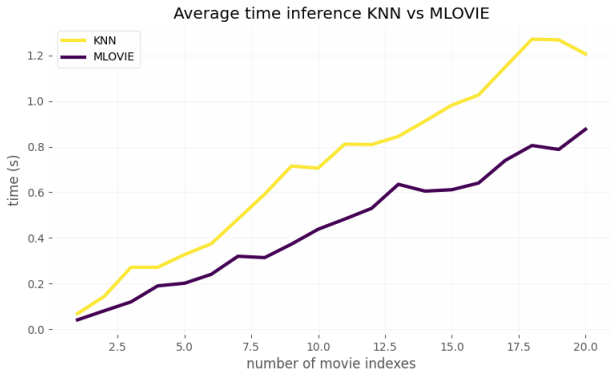

Figure 1: inference time on both models

The KNN (Embedded) model is consistently faster, and both models exhibit linear time growth. Although we set a limit of 5 movies for user input, the models can handle larger inputs without significant issues.

Key Findings

The KNN (Embedded) model demonstrated superior performance across all evaluation metrics:

- Achieved 30% **faster inference** by avoiding high-dimensional computations
- Enabled semantic analysis of textual features such as `description` and `Title`
- Produced more semantically coherent recommendations (validated via `t-SNE` and `UMAP` visualizations)
- Successfully integrated with backend through standardized interface
- Dimensionality of features:
  - KNN (one-hot): 2356 features (due to high cardinality from one-hot encoding)
  - KNN (Embedded): 9 features: 6 SBERT embeddings (each with 384 elements) plus 3 normalized numerical features, resulting in 2307 numerical values per observation

System Outcomes

- Enabled for both models real-time suggestions ($<$ 500 ms response time)
- Implemented a model-agnostic API supporting runtime switching:
  - Designed a polymorphic interface enabling runtime model switching
  - *Implementation*: Standardized `predict()` method across models.
  - *Benefit*: Frontend can toggle between KNN and Embedded models via a dropdown menu.

  Frontend can toggle between KNN/Embedded models via dropdown

Discussion & Conclusion

Technical Challenges

Several challenges emerged during development:

- **Heterogeneous Data**: TV shows and movies required different handling of duration
- **Semantic Gaps**: Plot descriptions often lacked thematic depth for accurate embeddings; more detailed descriptions would improve results in the KNN (Embedded) model
- **VAE Limitations**: Failed to learn meaningful latent representations from sparse features
- **Insufficient Metadata**: New movies without sufficient metadata remain challenging

System Limitations

The current implementation has several constraints:

- Static dataset (Netflix catalog as of 2021)
- Predictions are less accurate for movies with significant missing metadata (e.g. `ratings`, `listd_in`)
- No personalization beyond the initial seed movies
- Poor quality of movie descriptions

Conclusion

The developed recommendation system successfully addresses the core requirement of generating personalized movie suggestions from minimal user input. The KNN approach using sentence embeddings proved optimal, balancing performance with recommendation quality. The modular design allows seamless incorporation of future improvements.

Future Enhancements

- Incorporate lightweight user feedback mechanism
- Incorporate external APIs for richer metadata:
  - Richer descriptions could not only enable better movie recommendations but also allow search by user prompt descriptions
  - User score ratings could provide valuable information to improve recommendations; currently, this information is used only on the UI and not by any model
- Implement user feedback mechanism to improve recommendations

This project demonstrates that content-based filtering combined with modern NLP techniques provides effective recommendations while meeting the strict performance constraints required for web deployment. The MLOps approach ensured smooth integration between machine learning components and the production web environment.